

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Database Management Systems

COURSE CODE: CSA0509

COURSE OUTCOME COVERED

CO4: *Design database solutions incorporating file organization, indexing, and hashing techniques to optimize data storage and retrieval efficiency. (BL3, BL4)*

Activity Tool : Code Challenge (High)

Assessment Tool Weightage: 35%

Dr

QUESTIONS			Total Marks: 50
Q.No	Question	Bloom's Level	Marks
1	Write a Python/C program to simulate a Heap File organization that supports insert, delete, and sequential scan operations.	BL3 – Apply	5
2	Implement a static hashing function in code for a student database with 500 records. Handle collisions using chaining.	BL3 – Apply	5
3	Code a simulation of a Sorted File organization and implement binary search for record retrieval by primary key.	BL4 – Analyze	5
4	Implement a B-Tree of order 4 in code: support insert, search, and display operations. Insert keys: 10, 20, 5, 6, 12, 30, 7, 17.	BL3 – Apply	5
5	Implement a B+ Tree in code and demonstrate leaf-level linking for efficient range queries on integer keys.	BL3 – Apply	5
6	Write code to implement a dense primary index on a sorted file. Support search by index key and display the index table.	BL4 – Analyze	5
7	Implement extendible hashing in code. Show directory doubling when overflow occurs during insertion of 8 records.	BL3 – Apply	5

8	Write a program to simulate secondary storage block access. Calculate the number of I/O operations for sequential vs. indexed retrieval.	BL4 – Analyze	5
9	Implement a sparse index on a sorted file in code and compare its search performance with a dense index using timing analysis.	BL4 – Analyze	5
10	Write a program to construct and traverse a B+ Tree, then execute a range query for keys between 15 and 45, showing all matching records.	BL4 – Analyze	5

Code of Conduct:

I, Nisha R (192311415)(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Nisha R

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints
Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficient approach	No clear algorithmic approach
Code Implementation	20%	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect results

Competitive Performance	20%	Solves problem within time limits and handles large inputs effectively	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints
Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases

1. Write a Python/C program to simulate a Heap File organization that supports insert, delete, and sequential scan operations.

Aim:

To simulate Heap File organization supporting insertion, deletion, and sequential scanning of records.

Algorithm:

1. Store records in the order in which they are inserted.
2. Insert new records at the end of the heap file.
3. Delete a record by searching for its key.
4. Sequentially scan all available records.

Code:

```
class HeapFile:
    def __init__(self):
        self.records = []

    def insert(self, record):
        self.records.append(record)
        print("Inserted:", record)

    def delete(self, rollno):
        for record in self.records:
            if record[0] == rollno:
                self.records.remove(record)
                print("Deleted record:", record)
                return
        print("Record not found")

    def scan(self):
        print("\nSequential Scan:")
        for record in self.records:
            print(record)

heap = HeapFile()

heap.insert((101, "Arun"))
heap.insert((102, "Bala"))
heap.insert((103, "Cathy"))

heap.scan()

heap.delete(102)

heap.scan()
```

Output:

```
Enter "help" below or click "help" above for more information
>>>
===== RESTART: C:/Users/JUDY ALLEN/OneDrive/Desktop
Inserted: (101, 'Arun')
Inserted: (102, 'Bala')
Inserted: (103, 'Cathy')

Sequential Scan:
(101, 'Arun')
(102, 'Bala')
(103, 'Cathy')
Deleted record: (102, 'Bala')

Sequential Scan:
(101, 'Arun')
(103, 'Cathy')
>>> |
```

Complexity:

Insertion: $O(1)$

Deletion: $O(n)$

Sequential Scan: $O(n)$

2. Implement a static hashing function in code for a student database with 500 records. Handle collisions using chaining.

Aim:

To implement static hashing for student records and resolve collisions using chaining.

Algorithm:

1. Create a hash table of fixed size.
2. Calculate the bucket using key % table_size.
3. Store records having the same hash value in a list.
4. Thus, collisions are handled using chaining.

Code:

```
class StudentHash:
    def __init__(self, size=10):
        self.size = size
        self.table = [[] for _ in range(size)]

    def hash_function(self, student_id):
        return student_id % self.size

    def insert(self, student_id, name):
        index = self.hash_function(student_id)
        self.table[index].append((student_id, name))

    def display(self):
        for i in range(self.size):
            print(i, "->", self.table[i])

student_hash = StudentHash()

for i in range(1, 501):
    student_hash.insert(i, "Student" + str(i))

student_hash.display()
```

Output:

```
0 -> [(10, 'Student10'), (20, 'Student20'), (30, 'Student30'), (40, 'Student40'), (50, 'Student50'), (60, 'Student60'), (70, 'Student70'), (80, 'Student80'), (90, 'Student90'), (100, 'Student100'), (110, 'Student110'), (120, 'Student120'), (130, 'Student130'), (140, 'Student140'), (150, 'Student150'), (160, 'Student160'), (170, 'Student170'), (180, 'Student180'), (190, 'Student190'), (200, 'Student200'), (210, 'Student210'), (220, 'Student220'), (230, 'Student230'), (240, 'Student240'), (250, 'Student250'), (260, 'Student260'), (270, 'Student270'), (280, 'Student280'), (290, 'Student290'), (300, 'Student300'), (310, 'Student310'), (320, 'Student320'), (330, 'Student330'), (340, 'Student340'), (350, 'Student350'), (360, 'Student360'), (370, 'Student370'), (380, 'Student380'), (390, 'Student390'), (400, 'Student400'), (410, 'Student410'), (420, 'Student420'), (430, 'Student430'), (440, 'Student440'), (450, 'Student450'), (460, 'Student460'), (470, 'Student470'), (480, 'Student480'), (490, 'Student490'), (500, 'Student500')]
1 -> [(1, 'Student1'), (11, 'Student11'), (21, 'Student21'), (31, 'Student31'), (41, 'Student41'), (51, 'Student51'), (61, 'Student61'), (71, 'Student71'), (81, 'Student81'), (91, 'Student91'), (101, 'Student101'), (111, 'Student111'), (121, 'Student121'), (131, 'Student131'), (141, 'Student141'), (151, 'Student151'), (161, 'Student161'), (171, 'Student171'), (181, 'Student181'), (191, 'Student191'), (201, 'Student201'), (211, 'Student211'), (221, 'Student221'), (231, 'Student231'), (241, 'Student241'), (251, 'Student251'), (261, 'Student261'), (271, 'Student271'), (281, 'Student281'), (291, 'Student291'), (301, 'Student301'), (311, 'Student311'), (321, 'Student321'), (331, 'Student331'), (341, 'Student341'), (351, 'Student351'), (361, 'Student361'), (371, 'Student371'), (381, 'Student381'), (391, 'Student391'), (401, 'Student401'), (411, 'Student411'), (421, 'Student421'), (431, 'Student431'), (441, 'Student441'), (451, 'Student451'), (461, 'Student461'), (471, 'Student471'), (481, 'Student481'), (491, 'Student491')]
2 -> [(2, 'Student2'), (12, 'Student12'), (22, 'Student22'), (32, 'Student32'), (42, 'Student42'), (52, 'Student52'), (62, 'Student62'), (72, 'Student72'), (82, 'Student82'), (92, 'Student92'), (102, 'Student102'), (112, 'Student112'), (122, 'Student122'), (132, 'Student132'), (142, 'Student142'), (152, 'Student152'), (162, 'Student162'), (172, 'Student172'), (182, 'Student182'), (192, 'Student192'), (202, 'Student202'), (212, 'Student212'), (222, 'Student222'), (232, 'Student232'), (242, 'Student242'), (252, 'Student252'), (262, 'Student262'), (272, 'Student272'), (282, 'Student282'), (292, 'Student292'), (302, 'Student302'), (312, 'Student312'), (322, 'Student322'), (332, 'Student332'), (342, 'Student342'), (352, 'Student352'), (362, 'Student362'), (372, 'Student372'), (382, 'Student382'), (392, 'Student392'), (402, 'Student402'), (412, 'Student412'), (422, 'Student422'), (432, 'Student432'), (442, 'Student442'), (452, 'Student452'), (462, 'Student462'), (472, 'Student472'), (482, 'Student482'), (492, 'Student492')]
3 -> [(3, 'Student3'), (13, 'Student13'), (23, 'Student23'), (33, 'Student33'), (43, 'Student43'), (53, 'Student53'), (63, 'Student63'), (73, 'Student73'), (83, 'Student83'), (93, 'Student93'), (103, 'Student103'), (113, 'Student113'), (123, 'Student123'), (133, 'Student133'), (143, 'Student143'), (153, 'Student153'), (163, 'Student163'), (173, 'Student173'), (183, 'Student183'), (193, 'Student193'), (203, 'Student203'), (213, 'Student213'), (223, 'Student223'), (233, 'Student233'), (243, 'Student243'), (253, 'Student253'), (263, 'Student263'), (273, 'Student273'), (283, 'Student283'), (293, 'Student293'), (303, 'Student303'), (313, 'Student313'), (323, 'Student323'), (333, 'Student333'), (343, 'Student343'), (353, 'Student353'), (363, 'Student363'), (373, 'Student373'), (383, 'Student383'), (393, 'Student393'), (403, 'Student403'), (413, 'Student413'), (423, 'Student423'), (433, 'Student433'), (443, 'Student443'), (453, 'Student453'), (463, 'Student463'), (473, 'Student473'), (483, 'Student483'), (493, 'Student493')]
4 -> [(4, 'Student4'), (14, 'Student14'), (24, 'Student24'), (34, 'Student34'), (44, 'Student44'), (54, 'Student54'), (64, 'Student64'), (74, 'Student74'), (84, 'Student84'), (94, 'Student94'), (104, 'Student104'), (114, 'Student114'), (124, 'Student124'), (134, 'Student134'), (144, 'Student144'), (154, 'Student154'), (164, 'Student164'), (174, 'Student174'), (184, 'Student184'), (194, 'Student194'), (204, 'Student204'), (214, 'Student214'), (224, 'Student224'), (234, 'Student234'), (244, 'Student244'), (254, 'Student254'), (264, 'Student264'), (274, 'Student274'), (284, 'Student284'), (294, 'Student294'), (304, 'Student304'), (314, 'Student314'), (324, 'Student324'), (334, 'Student334'), (344, 'Student344'), (354, 'Student354'), (364, 'Student364'), (374, 'Student374'), (384, 'Student384'), (394, 'Student394'), (404, 'Student404'), (414, 'Student414'), (424, 'Student424'), (434, 'Student434'), (444, 'Student444'), (454, 'Student454'), (464, 'Student464'), (474, 'Student474'), (484, 'Student484'), (494, 'Student494')]
5 -> [(5, 'Student5'), (15, 'Student15'), (25, 'Student25'), (35, 'Student35'), (45, 'Student45'), (55, 'Student55'), (65, 'Student65'), (75, 'Student75'), (85, 'Student85'), (95, 'Student95'), (105, 'Student105'), (115, 'Student115'), (125, 'Student125'), (135, 'Student135'), (145, 'Student145'), (155, 'Student155'), (165, 'Student165'), (175, 'Student175'), (185, 'Student185'), (195, 'Student195'), (205, 'Student205'), (215, 'Student215'), (225, 'Student225'), (235, 'Student235'), (245, 'Student245'), (255, 'Student255'), (265, 'Student265'), (275, 'Student275'), (285, 'Student285'), (295, 'Student295'), (305, 'Student305'), (315, 'Student315'), (325, 'Student325'), (335, 'Student335'), (345, 'Student345'), (355, 'Student355'), (365, 'Student365'), (375, 'Student375'), (385, 'Student385'), (395, 'Student395'), (405, 'Student405'), (415, 'Student415'), (425, 'Student425'), (435, 'Student435'), (445, 'Student445'), (455, 'Student455'), (465, 'Student465'), (475, 'Student475'), (485, 'Student485'), (495, 'Student495')]
```

Since 500 records are inserted into only 10 buckets, multiple records enter the same bucket. These collisions are successfully handled using chaining.

Complexity:

Average insertion/search: $O(1)$

Worst case: $O(n)$

3. Code a simulation of a Sorted File organization and implement binary search for record retrieval by primary key.

Aim:

To simulate Sorted File organization and retrieve a record using binary search.

Code:

```
records = [
    (105, "David"),
    (101, "Arun"),
    (104, "Cathy"),
    (102, "Bala"),
    (103, "Ben")
]

records.sort(key=lambda x: x[0])

print("Sorted File:")
for record in records:
    print(record)

def binary_search(records, key):
    low = 0
    high = len(records) - 1

    while low <= high:
        mid = (low + high) // 2

        if records[mid][0] == key:
            return records[mid]

        elif records[mid][0] < key:
            low = mid + 1

        else:
            high = mid - 1

    return None

key = 104
result = binary_search(records, key)

if result:
    print("\nRecord Found:", result)
else:
```

Output:

```
Sorted File:
(101, 'Arun')
(102, 'Bala')
(103, 'Ben')
(104, 'Cathy')
(105, 'David')

Record Found: (104, 'Cathy')
|
```

Analysis:

Binary search repeatedly divides the search area into two halves. Hence it is considerably faster than sequential searching for large sorted files.

Time Complexity:

$O(\log n)$

4. Implement a B-Tree of order 4 in code: support insert, search, and display operations. Insert keys: 10, 20, 5, 6, 12, 30, 7, 17.

Aim:

To implement insertion, searching, and traversal of a B-Tree of order 4.

Code:

```
class BTreeNode:
    def __init__(self, leaf=False):
        self.keys = []
        self.children = []
        self.leaf = leaf

class BTree:
    def __init__(self):
        self.root = BTreeNode(True)
        self.t = 2

    def traverse(self, node=None):
        if node is None:
            node = self.root

        for i in range(len(node.keys)):
            if not node.leaf:
                self.traverse(node.children[i])

            print(node.keys[i], end=" ")

        if not node.leaf:
            self.traverse(node.children[len(node.keys)])

    def search(self, key, node=None):
        if node is None:
            node = self.root

        i = 0

        while i < len(node.keys) and key > node.keys[i]:
            i += 1

        if i < len(node.keys) and node.keys[i] == key:
            return True

        if node.leaf:
            return False
```

Output:

```
----- RESUME: C:/Users/000/
B-Tree Traversal:
5 6 7 10 12 17 20 30
Search 12: True
Search 50: False
|
```

5. Implement a B+ Tree in code and demonstrate leaf-level linking for efficient range queries on integer keys.

Aim:

To implement a simple B+ Tree and demonstrate linking between leaf nodes for range queries.

Code:

```
class Leaf:
    def __init__(self):
        self.keys = []
        self.next = None

leaf1 = Leaf()
leaf2 = Leaf()
leaf3 = Leaf()

leaf1.keys = [5, 10, 15]
leaf2.keys = [20, 25, 30]
leaf3.keys = [35, 40, 45]

leaf1.next = leaf2
leaf2.next = leaf3

def display_leaves(node):
    while node:
        print(node.keys, end=" -> ")
        node = node.next

    print("NULL")

def range_query(node, low, high):
    result = []

    while node:
        for key in node.keys:
            if low <= key <= high:
                result.append(key)

        node = node.next

    return result

print("Leaf Level Linking:")
```

```
Output:
Leaf Level Linking:
[5, 10, 15] -> [20, 25, 30] -> [35, 40, 45] -> NULL
Range Query 15 to 35:
[15, 20, 25, 30, 35]
```

```
Leaf Level Linking:
[5, 10, 15] -> [20, 25, 30] -> [35, 40, 45] -> NULL
Range Query 15 to 35:
[15, 20, 25, 30, 35]
```

Inference:
Leaf-level links allow B+ Trees to perform sequential and range retrieval efficiently without repeatedly returning to the root.

Leaf-level links allow B+ Trees to perform sequential and range retrieval efficiently without repeatedly returning to the root.

6. Write code to implement a dense primary index on a sorted file. Support search by index key and display the index table.

Aim:

To implement a dense primary index for a sorted file.

In a dense index, every record has a corresponding index entry.

Code:

```
records = [
    (101, "Arun"),
    (102, "Bala"),
    (103, "Cathy"),
    (104, "David"),
    (105, "Esha")
]

dense_index = {}

for position in range(len(records)):
    key = records[position][0]
    dense_index[key] = position

print("Dense Index Table")
print("Key\tAddress")

for key, address in dense_index.items():
    print(key, "\t", address)

search_key = 104

if search_key in dense_index:
    position = dense_index[search_key]
    print("\nRecord Found:", records[position])
else:
    print("\nRecord Not Found")|
```

Output:

```
Dense Index Table
Key      Address
101      0
102      1
103      2
104      3
105      4

Record Found: (104, 'David')
|
```

Analysis:

Dense indexing requires more index space because an entry exists for every record, but it provides fast direct retrieval.

7. Implement extendible hashing in code. Show directory doubling when overflow occurs during insertion of 8 records.

Aim:

To demonstrate extendible hashing and directory doubling when bucket overflow occurs while inserting eight records.

Code:

```
class ExtendibleHash:
    def __init__(self, bucket_size=2):
        self.global_depth = 1
        self.bucket_size = bucket_size
        self.directory = {
            "0": [],
            "1": []
        }

    def get_index(self, key):
        binary = bin(key)[2:]
        binary = binary.zfill(self.global_depth)
        return binary[-self.global_depth:]

    def double_directory(self):
        self.global_depth += 1

        old_records = []

        for bucket in self.directory.values():
            old_records.extend(bucket)

        self.directory = {}

        for i in range(2 ** self.global_depth):
            index = format(i, "0" + str(self.global_depth) + "b")
            self.directory[index] = []

        print("\nDirectory doubled.")
        print("Global Depth =", self.global_depth)

        for key in old_records:
            index = self.get_index(key)
            self.directory[index].append(key)

    def insert(self, key):
        index = self.get_index(key)

        if len(self.directory[index]) >= self.bucket_size:
            self.double_directory()
```

Output:

```
Directory doubled.  
Global Depth = 2
```

```
Directory:  
00 -> [20, 40]  
01 -> [5, 25]  
10 -> [10, 30]  
11 -> [15, 35]  
|
```

Analysis:

Whenever overflow occurs, the directory is expanded by increasing the global depth. This reduces collisions and allows the hash structure to grow dynamically.

8. Write a program to simulate secondary storage block access. Calculate the number of I/O operations for sequential vs. indexed retrieval.

Aim:

To calculate I/O operations required for sequential and indexed retrieval.

Code:

```
total_records = 1000
records_per_block = 10

total_blocks = total_records // records_per_block

# Worst-case sequential retrieval
sequential_io = total_blocks

# Assume index takes 2 I/O operations + 1 data block access
index_levels = 2
indexed_io = index_levels + 1

print("Total Records:", total_records)
print("Records per Block:", records_per_block)
print("Total Blocks:", total_blocks)

print("\nSequential Retrieval I/O:", sequential_io)
print("Indexed Retrieval I/O:", indexed_io)

print("\nI/O Operations Saved:",
      sequential_io - indexed_io)
```

Output:

```
Python 3.7.0 Shell: C:\Python37\Python
Total Records: 1000
Records per Block: 10
Total Blocks: 100

Sequential Retrieval I/O: 100
Indexed Retrieval I/O: 3

I/O Operations Saved: 97
```

Analysis:

Sequential retrieval may require checking all 100 blocks in the worst case, whereas indexed retrieval requires only a few block accesses. Therefore, indexing significantly reduces disk I/O.

9. Implement a sparse index on a sorted file in code and compare its search performance with a dense index using timing analysis.

Aim:

To implement a sparse index and compare its performance with a dense index using execution time.

Code:

```
import time

records = [(i, "Student" + str(i))
            for i in range(1, 10001)]

# Dense Index
dense_index = {}

for i in range(len(records)):
    dense_index[records[i][0]] = i

# Sparse Index
block_size = 100
sparse_index = {}

for i in range(0, len(records), block_size):
    sparse_index[records[i][0]] = i

search_key = 8750

# Dense Index Search
start = time.perf_counter()

position = dense_index.get(search_key)

dense_time = time.perf_counter() - start

# Sparse Index Search
start = time.perf_counter()

start_position = 0

for key in sparse_index:
    if key <= search_key:
        start_position = sparse_index[key]
    else:
```

Output:

```
===== RESTART: C:/Users/JUDY ALLEN/OneDr.  
Record: (8750, 'Student8750')  
  
Dense Index Entries: 10000  
Sparse Index Entries: 100  
  
Dense Search Time: 5.099980626255274e-06  
Sparse Search Time: 5.029997555539012e-05  
|
```

Comparison:

Dense Index	Sparse Index
One entry for every record	One entry for a block/group
Requires more storage	Requires less storage
Faster lookup	Slightly slower lookup
Direct record location	Block located first

Conclusion:

Dense indexing generally gives faster searching but requires more memory. Sparse indexing consumes less index space but may require an additional search within the selected data block.

10. Write a program to construct and traverse a B+ Tree, then execute a range query for keys between 15 and 45, showing all matching records.

Aim:

To construct and traverse a B+ Tree and retrieve all records between 15 and 45.

Code:

```
class LeafNode:
    def __init__(self):
        self.records = []
        self.next = None

leaf1 = LeafNode()
leaf2 = LeafNode()
leaf3 = LeafNode()

leaf1.records = [
    (5, "R1"),
    (10, "R2"),
    (15, "R3")
]

leaf2.records = [
    (20, "R4"),
    (25, "R5"),
    (30, "R6")
]

leaf3.records = [
    (35, "R7"),
    (40, "R8"),
    (45, "R9"),
    (50, "R10")
]

leaf1.next = leaf2
leaf2.next = leaf3

def traverse(node):
    print("B+ Tree Leaf Traversal:")

    while node:
        for record in node.records:
            print(record)
```

Output:

```
-----
B+ Tree Leaf Traversal:
```

```
(5, 'R1')
(10, 'R2')
(15, 'R3')
(20, 'R4')
(25, 'R5')
(30, 'R6')
(35, 'R7')
(40, 'R8')
(45, 'R9')
(50, 'R10')
```

```
Records between 15 and 45:
```

```
(15, 'R3')
(20, 'R4')
(25, 'R5')
(30, 'R6')
(35, 'R7')
(40, 'R8')
(45, 'R9')
```

Analysis:

B+ Trees store data entries at leaf nodes, and leaf nodes are linked sequentially. Once the first required key is located, subsequent keys can be retrieved by traversing the linked leaves, making range queries efficient.